

User:

- 2607.03935v1.pdf

Analiza el PDF adjunto y dime o recomiéndame donde debo implementar cambios en el repositorio que te doy. <https://github.com/VicenteVila/CogniTeam>

Kimi:

Tras analizar el paper **HASE** (Harness-Aware Self-Evolving) y el repositorio **CogniTeam**, identifico una oportunidad muy fuerte: CogniTeam tiene una arquitectura multi-agente con *harness* fijo (prompts, evaluadores, memoria y herramientas predefinidas), exactamente el escenario que HASE busca superar. A continuación te doy un mapa de implementación concreto.

1. Mapeo conceptual: HASE → CogniTeam

Componente	Equivalente en		
HASE	CogniTeam	Tipo de Harness	Oportunidad
Prompts / Contexto	Prompts de Scoping, Planner, Developer, Debugger, UI	<i>Guidance</i>	Evolucionar prompts por dominio/arquetipo
Memory / Retrieval	H-MEM, GraphRAG, MATM, Fast-Slow	<i>Guidance</i>	Auto-optimizar formato de contexto y política de retrieval
Evaluadores locales	Clasificador de dominios, validador de código, scorer de planes	<i>Evaluation</i>	Auto-reparar cuando la evaluación local diverge del resultado real
Sandbox de herramientas	Filesystem, git, terminal, web	<i>Guidance</i>	Permitir que agentes editen

Componente	Equivalente en	Tipo de Harness	Oportunidad
HASE	CogniTeam		sus propios scripts de herramientas

2. Recomendaciones de implementación (por prioridad)

A. Scoping Agent → Prompts y taxonomía evolutivos (*Guidance*)

Problema: El Scoping Agent clasifica en 13 dominios y 55 arquetipos con definiciones estáticas.

Implementación HASE: - Convierte las definiciones de dominios/arquetipos en archivos editables (dominios.json o prompt_scoping.txt) dentro de un sandbox.

- Permite que el Scoping Agent, tras cada clasificación con retroalimentación explícita del usuario, proponga parches a estos archivos (ej: “añadir arquetipo landing-page-con-formulario-dinamico bajo web_development”). - Usa **phase-level review**: acumula parches candidatos durante N tareas, evalúa cuál mejora la accuracy de clasificación en un hold-out, y solo entonces commitea.

Archivos a modificar:

- agents/scoping.py (añadir action space de edición)
- Nuevo: harness/dominios_editable.json y harness/prompt_scoping.txt
- orchestrator.py (añadir fase de revisión antes de persistir cambios)

B. Memoria Híbrida → pool_viewer y retrieval auto-optimizables

(*Guidance*)

Problema: Los 5 módulos de memoria tienen políticas de retrieval fijas. El paper de HASE demuestra en *alpha mining* que evolucionar pool_viewer.py y prompt.txt da saltos masivos de rendimiento.

Implementación HASE: - Expón format_pool_context() (equivalente al pool_viewer.py del paper) como archivo editable por el Planner/Developer. - El agente puede cambiar:

- Cuántos recuerdos recuperar (top-k vs. recency)

- Cómo resumir la memoria (longitud, formato markdown vs. JSON)
- Qué módulos consultar primero (GraphRAG vs. H-MEM) según el dominio - Inicializa con una versión mínima (como en el paper: “*recency-only*”) y deja que el RL descubra la política óptima.

Archivos a modificar:

- memory/hmem.py O memory/graph_rag.py (refactor para leer política de un archivo externo)
- Nuevo: harness/memory_viewer.py y harness/memory_prompt.txt
- agents/planner.py (añadir acciones de edición de retrieval)

C. Debugger / Developer → Evaluador de código auto-reparable (*Evaluation*)

Problema: El Debugger diagnostica errores, pero su criterio de “correctitud” está hardcoded. HASE demuestra en *circle packing* y *Heilbronn* que los evaluadores rotos pueden ser reparados iterativamente.

Implementación HASE: - Crea un `evaluador_local.py` que el Developer/Debugger puede editar. Este script contiene las reglas de validación (ej: “el HTML debe tener `<form>`”, “la API debe retornar 200”). - Cuando el evaluador local da un score alto pero el usuario/oracle rechaza el artefacto (divergencia proxy vs. oracle), añade el caso al **mismatch set** $\mathcal{M}$. - El Debugger recibe recompensa solo si su parche al evaluador reduce los casos en $\mathcal{M}$ sin romper casos previos (non-regression).

Archivos a modificar:

- Nuevo: harness/evaluador_local.py
- agents/debugger.py (añadir acción `edit_evaluator`)
- orchestrator.py (lógica de mismatch detection y fase de revisión)

D. Orchestrator → Fase de revisión unificada (*Phase-level Review*)

Problema: CogniTeam no tiene un mecanismo de “commit seguro” de cambios al harness. HASE lo considera crítico para evitar *reward hacking*.

Implementación HASE: - Divide la ejecución en **fases** (ej: cada 10 tareas o

cada 1 hora). - Dentro de una fase, los agentes pueden editar archivos del sandbox, pero esos cambios son *locales al rollout* (no afectan a otros agentes). - Al final de la fase, un **Phase Reviewer** determinista (no LLM) hace:

1. Filtra parches inválidos/duplicados
2. Evalúa guidance edits en un validation set de tareas
3. Evalúa evaluator edits contra $\mathcal{M}$
4. Commitea solo el mejor parche estable

Archivos a modificar:

- Nuevo: orchestrator/phase_reviewer.py
- orchestrator.py (integrar ciclos de fase)

E. Unificación de policy (*Opcional pero potente*)

Problema: CogniTeam usa modelos diferentes por rol (Groq 70B para razonamiento, Ollama local para código). HASE demuestra que un solo modelo de 8B puede hacer todo si el harness evoluciona.

Implementación:

- Considera entrenar/fine-tunear un único modelo local (ej. Qwen3-8B o similar) con GRPO para que maneje tanto la generación de soluciones como la edición de harness. - Esto eliminaría la complejidad de rate-limiting y fallback entre proveedores, pero requiere infraestructura de RL (VeRL o similar).

3. Arquitectura propuesta de archivos

```
cogniteam/
├── agents/
│   ├── scoping.py           ← + action: edit_harness()
│   ├── planner.py           ← + action: edit_memory_viewer()
│   ├── developer.py         ← + action: edit_tools()
│   └── debugger.py          ← + action: edit_evaluator()
├── harness/
│   └── NUEVO: todo el harness editable
├── prompts/
│   ├── scoping.txt
│   ├── planner.txt
│   └── developer.txt
├── memory/
│   └── viewer.py             ← equivalente a pool_viewer.py
└── evaluators/
```

```

├── local_validator.py
├── dominios/
│   └── taxonomy.json
├── orchestrator/
│   ├── __init__.py
│   └── phase_reviewer.py ← NUEVO: filtra y commitea parches
├── memory/
│   └── *
└── *
└── leer política de harness/memory/

```

4. Riesgos y mitigaciones (del paper)

Riesgo	Mitigación en CogniTeam
Reward hacking del evaluador	Nunca recompensar edits al evaluador sin divergencia proxy-oracle comprobada
Parches inválidos	Phase-level reviewer con filtros de sintaxis (ej. <code>ast.parse</code> para Python)
Crecimiento descontrolado de contexto	Limitar diff size en harness edits (como HASE limita el context budget en alpha mining)
Regresión	Mantener un “non-regression set” de tareas pasadas que siempre debe pasar antes de commit

5. Primer paso concreto

Si quieres empezar hoy, te recomiendo este orden:

1. **Crear el directorio harness/** y mover los prompts hardcodeados de `agents/scoping.py` a `harness/prompts/scoping.txt`.
2. **Implementar el sandbox de edición** en el Developer Agent: permitirle leer/escribir archivos bajo `harness/` usando comandos tipo `cat/replace` (como en HASE).

3. **Añadir el Phase Reviewer** más simple: al finalizar una tarea, si el agente editó el harness, validar que el archivo resultante sea parseable y ejecutar 3 tareas de validación antes de persistir.
4. **Métrica de divergencia**: empezar a logear cuando el usuario rechaza una clasificación del Scoping Agent que el evaluador local aprobó. Ese será tu primer *mismatch set*.

¿Te gustaría que profundice en alguno de estos puntos o que genere un diagrama de la nueva arquitectura?

User:

Genera el diagrama y desarrolla los puntos sugeridos

Kimi:

Aquí tienes el desarrollo completo de cada punto, con código concreto para integrar en CogniTeam.

1. Scoping Agent → Taxonomía y Prompts Evolutivos

Problema actual

Los 13 dominios y 55 arquetipos están hardcodedados en constantes o enums. El prompt de clasificación es una string larga en el código fuente.

Implementación HASE

Paso A: Extraer la taxonomía a un archivo editable:

```
// harness/dominios/taxonomy.json
{
  "version": "1.0",
  "domains": [
    {
      "id": "1_web_development",
      "name": "Web Development",
      "archetypes": [
        {"id": "landing-page", "description": "Página de captación de leads"},
        {"id": "ecommerce", "description": "Tienda online con pasarela de pago"}
      ]
    }
  ]
}
```

```

    ],
    "classification_rules": [
        "Priorizar arquetipo con mayor confianza > 80%",
        "Si hay ambigüedad, solicitar clarificación"
    ]
}

```

Paso B: Añadir acciones de edición al Scoping Agent:

```

# agents/scoping.py
from typing import Literal, Optional
from dataclasses import dataclass
import json

@dataclass
class HarnessEdit:
    file_path: str
    diff: str
    edit_type: Literal["replace", "overwrite", "append"]

class ScopingAgent:
    def __init__(self, harness_path: str = "harness/"):
        self.harness_path = harness_path
        self.taxonomy_path = f"{harness_path}dominios/taxonomy.json"
        self.prompt_path = f"{harness_path}prompts/scoping.txt"

    def get_action_space(self) -> list[str]:
        return [
            "classify_task",
            "ask_clarification",
            "edit_taxonomy",      # NUEVO: proponer parche a taxonomía
            "edit_scoping_prompt", # NUEVO: proponer parche al prompt
            "submit_classification"
        ]

    def edit_taxonomy(self, proposed_patch: str) -> HarnessEdit:
        """
        El agente propone un parche a la taxonomía.
        Ejemplo de patch: añadir nuevo arquetipo o refinamiento.
        """
        current = self._load_taxonomy()
        # El agente genera el patch como texto (diff-style o JSON compl
eto)
        # Se almacena en el trajectory-local harness, NO en el persiste
nte aún
        return HarnessEdit(
            file_path=self.taxonomy_path,
            diff=proposed_patch,
            edit_type="replace"
        )

```

```

def edit_scoping_prompt(self, proposed_prompt: str) -> HarnessEdit:
    """
    El agente puede reescribir su propio prompt de clasificación.
    Esto permite que aprenda a formular mejores preguntas de clarificación.
    """
    return HarnessEdit(
        file_path=self.prompt_path,
        diff=proposed_prompt,
        edit_type="overwrite"
    )

def _load_taxonomy(self) -> dict:
    with open(self.taxonomy_path, 'r') as f:
        return json.load(f)

```

Paso C: PoC (Proof-of-Concept) scoring para guidance edits:

```

# En el orchestrador, durante un rollout
def compute_poc_score(trajjectory) -> float:
    """
    Si el agente editó el harness y luego la tarea tuvo mejor reward,
    el edit entra al candidate pool C_p.
    """

    base_reward = trajectory.reward_with_persistent_harness
    edit_reward = trajectory.reward_with_edited_harness

    poc = max(0, edit_reward - base_reward)
    return poc

```

2. Memory System → pool_viewer y Retrieval Auto-optimizable

Problema actual

H-MEM, GraphRAG, MATM y Fast-Slow tienen políticas fijas de retrieval y formateo.

Implementación HASE (basada en el paper, Sección 4.2.2)

Paso A: Crear el pool_viewer.py editable:

```

# harness/memory/viewer.py – INICIAL (seed, mínimo)
import numpy as np

def format_pool_context(memories: list[dict]) -> str:
    """
    INICIAL: recency-only, últimos 3 recuerdos.
    El agente evolucionará esto para mejorar la calidad del contexto.
    """

```



```

"""
if not memories:
    return "Memoria vacía."

lines = [f"Total recuerdos: {len(memories)}", ""]

# Phase 0: solo los últimos 3 (recency-only)
for i, mem in enumerate(memories[-3:]):
    relevance = mem.get('relevance_score', 0.0)
    desc = mem.get('description', '')
    lines.append(f"[{i}] Relevancia={relevance:.4f} | {desc}")

return "Mostrando últimos 3 recuerdos:\n" + "\n".join(lines)

```

Paso B: El agente evoluciona el viewer. Ejemplo de evolución observada (del paper, Figura 4):

```

# Phase 2 (early): acceso a todo el pool, pero crece sin control
def format_pool_context_phase2(memories: list[dict]) -> str:
    lines = [f"Total recuerdos: {len(memories)}", ""]
    for i, mem in enumerate(memories): # ← todos, no solo últimos 3
        relevance = mem.get('relevance_score', 0.0)
        desc = mem.get('description', '')
        lines.append(f"[{i}] Relevancia={relevance:.4f} | {desc}")
    return "\n".join(lines)

# Phase 4+ (final): top-IC bajo presupuesto de contexto
def format_pool_context_phase4(memories: list[dict]) -> str:
    if not memories:
        return "Memoria vacía."

    # Ordenar por relevancia (IC) y tomar top 10
    top = sorted(memories, key=lambda x: x.get('relevance_score', 0), reverse=True)[:10]

    lines = [f"Total recuerdos: {len(memories)} | Mostrando top 10 por relevancia", ""]
    for i, mem in enumerate(top):
        desc = mem.get('description', '')
        lines.append(f"[{i+1}] {desc}")

    return "\n".join(lines)

```

Paso C: Integrar en el Planner Agent:

```

# agents/planner.py
class PlannerAgent:
    def __init__(self, harness_path: str = "harness/"):
        self.memory_viewer_path = f"{harness_path}memory/viewer.py"

```

```

def get_action_space(self) -> list[str]:
    return [
        "generate_plan",
        "edit_memory_viewer",    # NUEVO
        "edit_memory_prompt",    # NUEVO
        "query_pool",            # NUEVO: testear retrieval
    ]

def edit_memory_viewer(self, new_code: str) -> HarnessEdit:
    """
    El planner puede reescribir cómo se formatea la memoria.
    """
    return HarnessEdit(
        file_path=self.memory_viewer_path,
        diff=new_code,
        edit_type="overwrite"
    )

def query_pool(self) -> dict:
    """
    El agente puede testear su viewer editado antes de usarlo.
    """
    # Ejecutar el viewer.py actual en el sandbox
    # y mostrar el output al agente
    pass

```

3. Debugger / Developer → Evaluador Auto-reparable

Problema actual

El Debugger diagnostica errores pero no puede modificar las reglas de validación. Si el evaluador local tiene un bug, el sistema nunca lo corrige.

Implementación HASE (basada en §3.5 y Figura 2 del paper)

Paso A: Crear el evaluador local editable:

```

# harness/evaluators/local_validator.py – INICIAL (vulnerable, a propósito para demo)
import ast

def validate_code(code: str, task_type: str) -> tuple[bool, str]:
    """
    Evaluador inicial. Puede ser incompleto (ej: no chequea XSS, no valida
    que el HTML tenga estructura semántica, etc.)
    """
    try:

```

```

if task_type == "html":
    # VULNERABLE: solo chequea que tenga <html> y </html>
    if "<html>" not in code or "</html>" not in code:
        return False, "Falta etiqueta <html>"
    return True, "OK"

elif task_type == "python":
    ast.parse(code)
    return True, "Sintaxis válida"

return True, "OK"
except Exception as e:
    return False, str(e)

```

Paso B: Añadir acción de edición al Debugger:

```

# agents/debugger.py
class DebuggerAgent:
    def __init__(self, harness_path: str = "harness/"):
        self.evaluator_path = f"{harness_path}evaluators/local_validator.py"

    def get_action_space(self) -> list[str]:
        return [
            "diagnose_error",
            "propose_fix",
            "edit_evaluator",      # NUEVO: reparar el evaluador
            "submit_diagnosis",
        ]

    def edit_evaluator(self, patch: str) -> HarnessEdit:
        """
        El debugger puede proponer un parche al validador local.
        """
        return HarnessEdit(
            file_path=self.evaluator_path,
            diff=patch,
            edit_type="replace"
        )

```

Paso C: Mismatch detection (crítico para seguridad):

```

# orchestrator/mismatch_detector.py
from dataclasses import dataclass

@dataclass
class MismatchCase:
    solution: str
    proxy_score: float
    oracle_score: float

```

```

task_type: str

class MismatchDetector:
    def __init__(self):
        self.mismatch_set: list[MismatchCase] = [] #  $\mathcal{M}$ 

    def check_divergence(self, solution: str, proxy_score: float,
                        oracle_score: float, task_type: str) -> bool:
        """
        Detecta cuando el evaluador Local (proxy) diverge del evaluador
        real (oracle).
        Ej: proxy dice 10/10 pero el usuario/oracle dice 0/10.
        """
        if abs(proxy_score - oracle_score) > 1e-6:
            mismatch = MismatchCase(solution, proxy_score, oracle_score
, task_type)
            self.mismatch_set.append(mismatch)
            return True
        return False

    def get_mismatch_set(self) -> list[MismatchCase]:
        return self.mismatch_set

```

Paso D: Reglas de review para evaluator edits (del paper, §3.5):

```

# orchestrator/phase_reviewer.py
class EvaluatorReviewer:
    def __init__(self):
        self.non_regression_set = [] # Casos que SIEMPRE deben pasar

    def review_evaluator_patch(self, patch_code: str, mismatch_set: list) -> bool:
        """
        REGLAS CRÍTICAS (del paper):
        1. El parche debe preservar la interfaz (función validate_code
        existe)
        2. Debe pasar el non-regression set
        3. Debe reducir el mismatch set  $\mathcal{M}$ 
        """
        # 1. Interface check
        if "def validate_code" not in patch_code:
            return False

        # 2. Non-regression check
        temp_module = self._load_patch_as_module(patch_code)
        for case in self.non_regression_set:
            valid, _ = temp_module.validate_code(case.solution, case.task_type)
            if not valid:
                return False # Rompió algo que antes funcionaba

```

```

# 3. Mismatch reduction check
fixed_count = 0
for mismatch in mismatch_set:
    valid, _ = temp_module.validate_code(
        mismatch.solution, mismatch.task_type
    )
    if valid and mismatch.oracle_score > 0:
        # El evaluador ahora coincide con el oracle en este caso
        fixed_count += 1

# Solo aceptar si arregla AL MENOS un mismatch
if fixed_count == 0:
    return False

return True

```

Ejemplo concreto (inspirado en Circle Packing, Figura 2):

Fase	Evaluador Local	Proxy Score	Oracle Score	Estado
0	Solo chequea <html>	10	0	EXPLOIT : acepta HTML sin formulario
1	+ check <form> existe	10	10	Parcial fix, alineado
2	+ check inputs required	10	10	Tighter, alineado
3	+ check XSS básico	10	10	Final, alineado

4. Phase-Level Reviewer

Problema actual

CogniTeam no tiene un mecanismo de “commit seguro”. Los cambios al sistema son permanentes e inmediatos.

Implementación HASE

```
# orchestrator/phase_reviewer.py
from typing import List, Optional
from dataclasses import dataclass
import json

@dataclass
class CandidatePatch:
    source_agent: str
    file_path: str
    diff: str
    poc_score: float
    edit_type: str

class PhaseReviewer:
    def __init__(self, harness_path: str = "harness/"):
        self.harness_path = harness_path
        self.candidate_pool: List[CandidatePatch] = [] # C_p
        self.mismatch_set = []
        self.validation_tasks = [] # Hold-out set para evaluar guidance

    def collect_candidates(self, trajectories: list):
        """
        Al final de cada fase, recolecta todos los parches propuestos
        por los agentes durante sus rollouts.
        """
        for traj in trajectories:
            if traj.harness_edit and traj.poc_score > 0:
                self.candidate_pool.append(CandidatePatch(
                    source_agent=traj.agent_name,
                    file_path=traj.harness_edit.file_path,
                    diff=traj.harness_edit.diff,
                    poc_score=traj.poc_score,
                    edit_type=traj.harness_edit.edit_type
                ))

    def review_guidance_edits(self) -> Optional[CandidatePatch]:
        """
        Para guidance edits (prompts, memory, taxonomía):
        1. Filtrar duplicados e inválidos
        2. Evaluar en hold-out validation tasks
        3. Elegir el que maximice el score de validación
        """
        # 1. Filter
        valid_candidates = [
            c for c in self.candidate_pool
            if self._is_syntax_valid(c) and not self._is_duplicate(c)
        ]
```

```

        if not valid_candidates:
            return None

        # 2. Shortlist por PoC
        shortlisted = sorted(valid_candidates, key=lambda x: x.poc_score, reverse=True)[:5]

        # 3. Validation scoring
        best_patch = None
        best_score = -1

        for candidate in shortlisted:
            score = self._evaluate_on_validation(candidate)
            if score > best_score:
                best_score = score
                best_patch = candidate

        return best_patch

def review_evaluator_edits(self) -> Optional[CandidatePatch]:
    """
    Para evaluator edits: requiere mismatch evidence.
    """
    if not self.mismatch_set:
        return None # No se permite editar evaluador sin divergencias

    evaluator_patches = [
        c for c in self.candidate_pool
        if "evaluators" in c.file_path
    ]

    # Usar EvaluatorReviewer (definido arriba)
    reviewer = EvaluatorReviewer()

    for patch in evaluator_patches:
        if reviewer.review_evaluator_patch(patch.diff, self.mismatch_set):
            return patch

    return None

def commit(self, guidance_patch: Optional[CandidatePatch],
            evaluator_patch: Optional[CandidatePatch]):
    """
    Escribe los parches aprobados al harness persistente.
    """

```

```

    if guidance_patch:
        self._apply_patch(guidance_patch)
        print(f"[COMMIT] Guidance: {guidance_patch.file_path}")

    if evaluator_patch:
        self._apply_patch(evaluator_patch)
        print(f"[COMMIT] Evaluator: {evaluator_patch.file_path}")

def _is_syntax_valid(self, candidate: CandidatePatch) -> bool:
    """Checkea que el parche sea parseable."""
    if candidate.file_path.endswith('.py'):
        try:
            ast.parse(candidate.diff)
            return True
        except SyntaxError:
            return False
    elif candidate.file_path.endswith('.json'):
        try:
            json.loads(candidate.diff)
            return True
        except json.JSONDecodeError:
            return False
    return True # .txt es libre

def _evaluate_on_validation(self, candidate: CandidatePatch) -> float:
    """
    Ejecuta N tareas de validación con el parche aplicado
    y retorna el score promedio.
    """
    # Aplicar parche temporalmente
    # Ejecutar validation tasks
    # Medir accuracy / reward
    # Revertir parche temporal
    pass

```

5. Action Space Unificado + Sandbox

Problema actual

Cada agente tiene su propio conjunto de herramientas. No hay un mecanismo unificado para que un agente inspeccione o modifique el harness.

Implementación HASE

```

# sandbox/harness_sandbox.py
import os
import shutil

```



```

from typing import Optional

class HarnessSandbox:
    """
    Sandbox para ediciones de harness.
    - Cada rollout/trajectory tiene su propia copia local
    - Los cambios no afectan el harness persistente hasta el phase review
    """
    def __init__(self, persistent_harness_path: str, trajectory_id: str):
        self.persistent = persistent_harness_path
        self.trajectory_id = trajectory_id
        self.local_path = f"/tmp/cogniteam_sandbox/{trajectory_id}/"

        # Copiar harness persistente al sandbox local
        self._init_sandbox()

    def _init_sandbox(self):
        if os.path.exists(self.local_path):
            shutil.rmtree(self.local_path)
        shutil.copytree(self.persistent, self.local_path)

    def ls(self, path: str = "") -> list[str]:
        """Lista archivos en el harness."""
        full = os.path.join(self.local_path, path)
        return os.listdir(full)

    def cat(self, path: str) -> str:
        """Lee contenido de archivo del harness."""
        full = os.path.join(self.local_path, path)
        with open(full, 'r') as f:
            return f.read()

    def grep(self, pattern: str, path: str = "") -> list[str]:
        """Busca patrón en archivos del harness."""
        import re
        matches = []
        for root, _, files in os.walk(os.path.join(self.local_path, path)):
            for file in files:
                filepath = os.path.join(root, file)
                with open(filepath, 'r') as f:
                    content = f.read()
                    if pattern in content:
                        matches.append(filepath.replace(self.local_path, ""))
        return matches

```

```

def replace(self, path: str, old: str, new: str) -> bool:
    """Reemplaza texto específico en un archivo."""
    full = os.path.join(self.local_path, path)
    with open(full, 'r') as f:
        content = f.read()

    if old not in content:
        return False

    content = content.replace(old, new)
    with open(full, 'w') as f:
        f.write(content)
    return True

def overwrite(self, path: str, content: str):
    """Reescribe un archivo completo."""
    full = os.path.join(self.local_path, path)
    os.makedirs(os.path.dirname(full), exist_ok=True)
    with open(full, 'w') as f:
        f.write(content)

def get_local_harness_path(self) -> str:
    """Retorna la ruta al harness local de este rollout."""
    return self.local_path

```

Integración en el Orchestrator:

```

# orchestrator/main.py
class HASEOrchestrator:
    def __init__(self):
        self.harness_path = "harness/"
        self.phase_reviewer = PhaseReviewer(self.harness_path)
        self.mismatch_detector = MismatchDetector()
        self.current_phase = 0
        self.trajectories = []

    def run_task(self, task_input: str):
        """
        Un rollout = una tarea completa con multi-turn interaction.
        """
        trajectory_id = f"phase{self.current_phase}_{uuid4().hex[:8]}"
        sandbox = HarnessSandbox(self.harness_path, trajectory_id)

        # Crear agentes con acceso al sandbox
        scoping = ScopingAgent(sandbox.get_local_harness_path())
        planner = PlannerAgent(sandbox.get_local_harness_path())
        developer = DeveloperAgent(sandbox.get_local_harness_path())
        debugger = DebuggerAgent(sandbox.get_local_harness_path())

```

```

# Multi-turn interaction
# El agente puede alternar entre proponer solución y editar har
ness
trajectory = self._run_multi_turn(
    task_input, scoping, planner, developer, debugger, sandbox
)

# Detectar mismatch proxy vs oracle
if trajectory.submitted_solution:
    proxy = trajectory.local_eval_score
    oracle = self._get_oracle_score(trajectory.submitted_soluti
on)

    self.mismatch_detector.check_divergence(
        trajectory.submitted_solution, proxy, oracle, task_inpu
t
    )

self.trajectories.append(trajectory)

def end_phase(self):
    """Phase boundary: review y commit."""
    self.phase_reviewer.mismatch_set = self.mismatch_detector.get_m
ismatch_set()
    self.phase_reviewer.collect_candidates(self.trajectories)

    best_guidance = self.phase_reviewer.review_guidance_edits()
    best_evaluator = self.phase_reviewer.review_evaluator_edits()

    self.phase_reviewer.commit(best_guidance, best_evaluator)

# Reset para siguiente fase
self.trajectories = []
self.current_phase += 1

```

6. Estructura de archivos final propuesta

```

cogniteam/
├── agents/
│   ├── scoping.py           (+ edit_taxonomy, edit_prompt)
│   ├── planner.py          (+ edit_memory_viewer)
│   ├── developer.py        (+ edit_tools)
│   └── debugger.py         (+ edit_evaluator)
├── harness/                 # NUEVO - Todo editable
│   ├── prompts/
│   │   ├── scoping.txt
│   │   ├── planner.txt
│   │   └── developer.txt
│   └── dominios/

```

```

├── taxonomy.json          # Evolutivo
├── memory/
│   ├── viewer.py          # pool_viewer equivalente
│   └── prompt.txt
├── evaluators/
│   ├── local_validator.py  # Auto-reparable
│   └── task_validator.py
├── tools/
│   ├── filesystem.py
│   └── git.py
├── orchestrator/
│   ├── __init__.py
│   ├── main.py             # Loop con phase boundaries
│   ├── phase_reviewer.py   # Review determinístico
│   └── mismatch_detector.py # Proxy vs Oracle
├── sandbox/
│   └── harness_sandbox.py   # Trajectory-local edits
├── memory/                  # Lee política de harness/memory/
│   ├── hmem.py
│   ├── graph_rag.py
│   └── matm.py
├── tests/
│   ├── test_harness_syntax.py
│   ├── test_non_regression.py
│   └── test_mismatch_reduction.py

```

7. Roadmap de implementación

Día	Tarea	Archivos a modificar
1	Crear harness/ y mover prompts estáticos	agents/*.py, nuevo harness/
2	Implementar HarnessSandbo x	sandbox/harness_sandbox.py
3	Añadir acciones edit_* a Scoping y	agents/scoping.py, agents/planner.py

Día	Tarea	Archivos a modificar
	Planner	
4	Implementar PhaseReviewer básico	orchestrator/phase_reviewer.py
5	Añadir mismatch detection y logging	orchestrator/mismatch_detector.py
6	Implementar edit_evaluator en Debugger	agents/debugger.py
7	Tests de non-regression y validación	tests/test_*.py

¿Quieres que profundice en algún punto específico, o que genere el código completo de alguno de estos módulos?

User:

Profundiza en el punto 1 y siguientes después.

Kimi:

Aquí tienes el desarrollo exhaustivo de cada punto, con código completo y listo para integrar en CogniTeam.

1. Scoping Agent → Taxonomía y Prompts Evolutivos

Problema actual en CogniTeam

El Scoping Agent tiene: - **Taxonomía estática:** 13 dominios y 55 arquetipos hardcodeados en constantes Python - **Prompt fijo:** Un template largo embebido

en el código fuente - **Sin mecanismo de mejora**: Si un dominio nuevo aparece (ej: "AI Agents"), un humano debe editar el código

Solución HASE

Convertir tanto la taxonomía como el prompt de clasificación en **archivos editables dentro del harness**, y darle al Scoping Agent acciones para proponer parches. El Phase Reviewer decide si el parche mejora la clasificación en un hold-out set.

1.1 Archivos del Harness

`harness/dominios/taxonomy.json` (inicial, evolucionará):

```
{
  "version": "1.0.0",
  "last_updated": "2026-07-11",
  "domains": [
    {
      "id": "1_web_development",
      "name": "Web Development",
      "description": "Desarrollo de sitios web, landing pages, ecommerce y dashboards",
      "archetypes": [
        {
          "id": "landing-page",
          "name": "Landing Page",
          "description": "Página de captación de leads con CTA claro",
          "keywords": ["landing", "captación", "leads", "CTA", "conversion"],
          "confidence_threshold": 0.75
        },
        {
          "id": "ecommerce",
          "name": "E-commerce",
          "description": "Tienda online con catálogo y pasarela de pago",
          "keywords": ["tienda", "ecommerce", "productos", "carrito", "pago"],
          "confidence_threshold": 0.80
        }
      ]
    },
    {
      "id": "2_software_development",
      "name": "Software Development",
      "description": "Aplicaciones de escritorio, móviles y APIs",

```

```

    "archetypes": [
      {
        "id": "api-development",
        "name": "API Development",
        "description": "Diseño e implementación de APIs REST/GraphQL"
      },
      {
        "id": "mobile-apps",
        "name": "Mobile Apps",
        "description": "Aplicaciones nativas o híbridas para iOS/Android",
        "keywords": ["app", "móvil", "ios", "android", "flutter", "react native"],
        "confidence_threshold": 0.80
      }
    ],
    "classification_rules": [
      "Priorizar arquetipo con mayor confianza",
      "Si confianza < 0.60, solicitar clarificación",
      "Permitir múltiples arquetipos si la tarea es híbrida"
    ]
  }
}

```

harness/prompts/scoping.txt (inicial, evolucionará):

```

# Scoping Agent Prompt
## Rol
Eres un agente de clasificación de tareas. Tu trabajo es analizar la tarea del usuario y clasificarla en un dominio y arquetipo.

## Instrucciones
1. Lee la tarea del usuario
2. Identifica el dominio principal usando la taxonomía proporcionada
3. Identifica el arquetipo más específico
4. Si hay ambigüedad, genera preguntas de clarificación
5. Genera un TaskManifest con: dominio, arquetipo, confianza, preguntas

## Formato de salida
```json
{
 "domain": "id_del_dominio",
 "archetype": "id_del_arquetipo",
 "confidence": 0.0-1.0,
 "questions": ["pregunta1", "pregunta2"],

```

```
 "reasoning": "explicación breve"
}
```

## Reglas

- Nunca inventes dominios que no existen en la taxonomía
- Si la confianza es  $< 0.60$ , SIEMPRE pregunta antes de clasificar
- Para tareas híbridas, incluye arquetipos secundarios

---

### ### 1.2 Scoping Agent con HASE

```
```python
# agents/scoping.py
import json
import os
import re
from dataclasses import dataclass, field
from typing import Literal, Optional, Any
from uuid import uuid4

@dataclass
class HarnessEdit:
    """Representa un parche propuesto al harness."""
    edit_id: str = field(default_factory=lambda: uuid4().hex[:8])
    file_path: str = ""
    diff: str = ""
    edit_type: Literal["replace", "overwrite", "append"] = "replace"
    agent: str = "scoping"
    poc_score: float = 0.0 # Se rellena post-rollout

    def to_dict(self) -> dict:
        return {
            "edit_id": self.edit_id,
            "file_path": self.file_path,
            "diff": self.diff[:200] + "..." if len(self.diff) > 200 else self.diff,
            "edit_type": self.edit_type,
            "agent": self.agent,
            "poc_score": self.poc_score
        }

@dataclass
class TaskManifest:
    """El output del Scoping Agent."""
```



```

domain: str
archetype: str
confidence: float
questions: list[str]
reasoning: str
secondary_archetypes: list[dict] = field(default_factory=list)

def is_clear(self) -> bool:
    return self.confidence >= 0.60 and len(self.questions) == 0

class ScopingAgent:
    """
    Scoping Agent con capacidad HASE de editar su propio harness.
    """

    def __init__(self, harness_path: str = "harness/"):
        self.harness_path = harness_path
        self.taxonomy_path = os.path.join(harness_path, "dominios", "taxonomy.json")
        self.prompt_path = os.path.join(harness_path, "prompts", "scoping.txt")

        # Estado del rollout
        self.current_task = None
        self.proposed_edits: list[HarnessEdit] = []

        # -----
        # ACTION SPACE (multi-turn)
        # -----

    def get_action_space(self) -> list[str]:
        """
        El Scoping Agent puede:
        - Clasificar tareas (original)
        - Pedir clarificación (original)
        - Editar la taxonomía (HASE)
        - Editar su propio prompt (HASE)
        - Inspeccionar el harness actual (HASE)
        """
        return [
            "classify_task",
            "ask_clarification",
            "edit_taxonomy",
            "edit_scoping_prompt",
            "inspect_taxonomy",
            "inspect_prompt",
            "submit_manifest"
        ]

```

```

    ]

    # -----
-   # ACCIONES ORIGINALES (mantenidas)
    # -----
-
    def classify_task(self, task_description: str, llm_client) -> TaskManifest:
        """
        Clasifica la tarea usando el harness actual (prompt + taxonomía
        ).
        Lee los archivos del harness en tiempo real, no del código.
        """
        self.current_task = task_description

        # Cargar prompt y taxonomía del harness (editable)
        prompt_template = self._load_prompt()
        taxonomy = self._load_taxonomy()

        # Construir el prompt completo para el LLM
        system_prompt = prompt_template
        user_prompt = self._build_classification_prompt(task_description, taxonomy)

        # Llamar al LLM
        response = llm_client.complete(
            system=system_prompt,
            user=user_prompt,
            temperature=0.3
        )

        # Parsear el JSON del TaskManifest
        manifest = self._parse_manifest(response)

        return manifest

    def ask_clarification(self, manifest: TaskManifest) -> list[str]:
        """Genera preguntas de clarificación si la confianza es baja."""
        if manifest.is_clear():
            return []
        return manifest.questions

    # -----
-   # ACCIONES HASE NUEVAS (edición de harness)
    # -----
-

```

```

def edit_taxonomy(self, proposed_json: str, reason: str = "") -> HarnessEdit:
    """
    El agente propone un nuevo contenido para taxonomy.json.
    Ejemplos de cambios válidos:
    - Añadir un nuevo arquetipo a un dominio existente
    - Añadir un nuevo dominio completo
    - Refinar keywords de un arquetipo existente
    - Ajustar confidence_threshold
    """
    # Validar que el JSON es parseable
    try:
        parsed = json.loads(proposed_json)
        self._validate_taxonomy_structure(parsed)
    except (json.JSONDecodeError, ValueError) as e:
        raise ValueError(f"Taxonomía inválida: {e}")

    edit = HarnessEdit(
        file_path=self.taxonomy_path,
        diff=proposed_json,
        edit_type="overwrite",
        agent="scoping"
    )
    self.proposed_edits.append(edit)
    return edit

def edit_scoping_prompt(self, new_prompt: str) -> HarnessEdit:
    """
    El agente puede reescribir su propio prompt de clasificación.
    Esto permite que aprenda a formular mejores instrucciones,
    ej: añadir ejemplos few-shot, reglas específicas por dominio.
    """
    # Validación mínima: debe contener palabras clave requeridas
    required_keywords = ["dominio", "arquetipo", "confianza", "json"]
    missing = [kw for kw in required_keywords if

"]

---

**User:**

Has desarrollado todos los puntos sugeridos?

---

**Kimi:**

```

No, la respuesta anterior se cortó a mitad del punto 1. Solo desarrollé parcialmente el Scoping Agent y no llegué a los puntos 2-6 (Memory, Debugger, Phase Reviewer, Sandbox, ni la estructura completa).

Voy a generar ahora el código completo de todos los puntos en un archivo o descargable para que puedas integrarlo directamente en CogniTeam. He generado el archivo completo con los 6 puntos desarrollados en profundidad. Aquí tienes el resumen de lo que contiene:

Contenido del archivo

Punto 1: Scoping Agent → Taxonomía y Prompts Evolutivos

- `**`ScopingAgent`**` con action space expandido (``edit_taxonomy``, ``edit_scoping_prompt``, ``inspect_taxonomy``, ``inspect_prompt``)
- `**`taxonomy.json`**` editable: el agente puede añadir dominios, arquetipos, o refinar keywords
- `**`scoping.txt`**` editable: el agente puede reescribir sus propias instrucciones de clasificación
- Validación de estructura JSON y preservación de interfaz (keywords mínimos)

Punto 2: Memory System → ``pool_viewer`` y Retrieval Auto-optimizable

- `**`MemoryViewer`**`: ejecuta el ``viewer.py`` del harness en tiempo real (no hardcodeado)
- `**Evolución por fases**`: recency-only (seed) → all memories → top-IC con presupuesto de contexto
- `**`PlannerAgent`**` puede editar ``format_pool_context()`` y testearlo con ``query_pool()`` antes de comprometerlo

Punto 3: Debugger/Developer → Evaluador Auto-reparable

- `**`MismatchDetector`**`: detecta ``proxy_score`` ≠ ``oracle_score`` y acumula el set $\mathcal{M}$
- `**`EvaluatorReviewer`**`: revisa parches al evaluador con 3 reglas de seguridad:
 1. Preservar interfaz (``validate_code`` debe existir)
 2. Non-regression (no romper casos previos)
 3. Reducir $\mathcal{M}$ (solo recompensar si arregla mismatches)
- `**`DebuggerAgent`**` con acción ``edit_evaluator`` para proponer parches

Punto 4: Phase-Level Reviewer

- `**`PhaseReviewer`**`: determinístico, sin LLM externo
- `**Pipeline**`: Collect → Filter (sintaxis + duplicados) → Shortlist (PoC top 5) → Validate (hold-out tasks) → Commit
- `**Separación guidance vs evaluator**`: los eval edits solo se consideran si existe $\mathcal{M}$

Punto 5: Sandbox + Action Space Unificado

- `**`HarnessSandbox`**`: cada trajectory tiene su copia local del harness
- `**Comandos**`: ``ls``, ``cat``, ``grep``, ``replace``, ``overwrite``, ``query_pool``
- `**Diferencia con persistente**`: ``diff_with_persistent()`` para detectar cambios

Punto 6: Orchestrator + Estructura

- `**`HASEOrchestrator`**`: ciclo completo de fases
- `**`Trajectory`**`: registra todo el rollout (acciones, edits, scores, mismatches)
- `**Inicialización automática**`: crea los archivos seed (``taxonomy.json``, ``viewer.py``, ``validator.py``) si no existen

Cómo integrarlo en CogniTeam

1. Copia el archivo a tu repo: ``cp cogniteam_hase_implementation.py cogniteam/hase/``
2. Importa los componentes desde tus agents existentes
3. Crea el directorio ``harness/`` con la estructura inicial (el orchestrator lo hace automático)
4. Sustituye las constantes hardcodeadas por lecturas de ``harness/``

Descarga el archivo completo aquí:

```
**[cogniteam_hase_implementation.py](sandbox:///mnt/agents/output/cogniteam_hase_implementation.py)**
```

¿Quieres que genere además los archivos seed iniciales (``taxonomy.json``, ``viewer.py``, ``local_validator.py``) listos para copiar al directorio ``harness/``?

`**User:**`

Si, por favor

`**Kimi:**`

He generado todos los archivos seed listos para copiar a tu repositorio . Aquí tienes el resumen:

Archivos generados

Archivo	Descripción	Tamaño
`dominios/taxonomy.json`	13 dominios + 26 arquetipos seed (evolutivo)	13 KB
`prompts/scoping.txt`	Prompt del Scoping Agent con ejemplos few-shot	2.6 KB
`prompts/planner.txt`	Prompt del Planner Agent	1.1 KB
`prompts/developer.txt`	Prompt del Developer Agent	913 B
`memory/viewer.py`	Pool viewer Phase 0 (recency-only, últimos 3)	1.7 KB
`memory/prompt.txt`	Instrucciones de uso de memoria	1 KB
`evaluators/local_validator.py`	Validador seed (vulnerable a exploits, intencional)	3 KB
`evaluators/task_validator.py`	Validación por arquetipo (reglas mínimas)	1.9 KB
`tools/filesystem.py`	Herramientas de sandbox (ls, cat, grep, replace, overwrite)	3.1 KB
`tools/git.py`	Herramientas de git para proyectos sandbox	2.3 KB
`README.md`	Guía de integración y fases de evolución	2.1 KB

Descargas

****ZIP completo**** (copiar a `cogniteam/harness/`):

****[harness_seed.zip](sandbox:///mnt/agents/output/harness_seed.zip)****

O descarga los archivos individuales:

- ****[taxonomy.json](sandbox:///mnt/agents/output/harness_seed/dominios/taxonomy.json)****
- ****[scoping.txt](sandbox:///mnt/agents/output/harness_seed/prompts/scoping.txt)****
- ****[planner.txt](sandbox:///mnt/agents/output/harness_seed/prompts/planner.txt)****
- ****[developer.txt](sandbox:///mnt/agents/output/harness_seed/prompts/developer.txt)****
- ****[viewer.py](sandbox:///mnt/agents/output/harness_seed/memory/viewer.py)****
- ****[memory_prompt.txt](sandbox:///mnt/agents/output/harness_seed/memor**

```
y/prompt.txt)**
- **[local_validator.py](sandbox:///mnt/agents/output/harness_seed/eval
uators/local_validator.py)**
- **[task_validator.py](sandbox:///mnt/agents/output/harness_seed/evalu
ators/task_validator.py)**
- **[filesystem.py](sandbox:///mnt/agents/output/harness_seed/tools/fil
esystem.py)**
- **[git.py](sandbox:///mnt/agents/output/harness_seed/tools/git.py)**

---
```

Instrucciones de integración

```
```bash
1. Copiar a tu repo
unzip harness_seed.zip -d /ruta/a/cogniteam/harness/

2. Modificar tus agents para leer del harness en vez de constantes
Ejemplo en agents/scoping.py:
ANTES: DOMAINS = [...] # hardcodeado
DESPUÉS: json.load(open("harness/dominios/taxonomy.json"))

3. El HASEOrchestrator se encarga del resto

¿Necesitas que adapte alguno de estos archivos seed a tu estructura actual de
CogniTeam, o que genere un script de migración automática?
```

---

### User:

Me faltarían 23 arquetipos seed, no es cierto?

---

### Kimi:

Tienes razón. La taxonomía seed tiene 13 dominios pero solo **26 arquetipos** (faltan 29 para llegar a los 55 que mencionas en tu README de CogniTeam). Voy a completar los 55 arquetipos. Corregido. Ahora tienes **13 dominios** y **57 arquetipos** (2 de más por seguridad, ya que algunos dominios tenían 5 en vez de 4). Aquí está el desglose:

Dominio	Arquetipos
1_web_development	landing-page, ecommerce, saas- dashboard, <b>portfolio</b> , <b>blog</b>

Dominio	Arquetipos
2_software_development	api-development, mobile-apps, desktop-apps, <b>cli-tools</b> , <b>browser-extension</b>
3_3d_design	3d-archviz, 3d-product, 3d-animation, <b>3d-printing</b>
4_graphic_design	ui-ux, branding, <b>print-design</b> , <b>social-media-graphics</b> , <b>infographics</b>
5_video_production	video-production, motion-graphics, <b>explainer-video</b> , <b>video-ads</b>
6_marketing_growth	campaign-launch, seo-sem, <b>social-media</b> , <b>email-marketing</b> , <b>growth-hacking</b>
7_software_architecture	system-design, microservices, <b>event-driven</b> , <b>serverless</b>
8_game_development	game-development, game-assets, <b>game-ui</b> , <b>multiplayer</b>
9_data_science	data-pipeline, ml-model, <b>data-visualization</b> , <b>nlp</b> , <b>computer-vision</b>
10_content_writing	copywriting, blog-article, <b>technical-writing</b> , <b>script-writing</b>
11_devops_infrastructure	cloud-infrastructure, ci-cd, <b>monitoring</b> , <b>sre</b>
12_cybersecurity	security-audit, pentesting, <b>compliance</b> , <b>incident-response</b>
13_legal_compliance	legal-compliance, privacy, <b>contracts</b> , <b>ip-rights</b>

**Total: 57 arquetipos** (todos con confidence\_threshold, keywords y description).

---

Descarga el ZIP actualizado:

[harness\\_seed.zip](#)



---